These problems are about the kernel you built in `34k_processes`, `35k_context_switch` and `36k_scheduling`. There are 3 problems on 5 pages. Answer each part in the space below it, and show your work.

1. **Processes (34k).** Here are `allocproc` and `freeproc` from exercise 34k. `alloc_pid` returns `NEXTPID` and increments it; it starts at 1 after `init()`.

```
pub unsafe fn allocproc() -> *mut Proc {
    for i in 0..NPROC {
        let p = ptr::addr_of_mut!(PROCS[i]);
        if (*p).state == ProcState::Unused {
            (*p).pid = alloc_pid();
            (*p).state = ProcState::Runnable;
            (*p).pagetable = create_pagetable();
            if (*p).pagetable.is_null() {
                return ptr::null_mut();
            }
            return p;
        }
    }
    ptr::null_mut()
}
```

```
pub unsafe fn freeproc(p: *mut Proc) {
    free_pagetable((*p).pagetable);
    (*p).pagetable = ptr::null_mut();
    (*p).pid = 0;
    (*p).state = ProcState::Unused;
}

// NPROC = 4, after init():
let a = allocproc();   // 1
let b = allocproc();   // 2
let c = allocproc();   // 3
freeproc(b);           // 4
let d = allocproc();   // 5
let e = allocproc();   // 6
let f = allocproc();   // 7
```

(a) For `d`, `e` and `f`, give the table slot and pid it refers to (or say it is null). Then give the state and pid of all four slots after line 7.

Lines 1–3 fill slots 0, 1, 2 with pids 1, 2, 3. Line 4 frees slot 1 (Unused, pid 0). NEXTPID is 4 and is never wound back.

d → slot 1, pid 4 (first Unused slot; pids are not reused)

e → slot 3, pid 5

f → null (all NPROC = 4 slots are taken)

After line 7: slot 0 Runnable pid 1, slot 1 Runnable pid 4, slot 2 Runnable pid 3, slot 3 Runnable pid 5.

(b) Now suppose instead the kernel is out of memory at line 6, so `create_pagetable()` returns null. What does `allocproc` return, and what is left in the slot it chose? Why is that a problem for the rest of the kernel? Fix `allocproc`.

It returns null, so the caller thinks nothing was allocated. But slot 3 was already claimed: state = Runnable, pid = 5, pagetable = null.

Nobody holds a pointer to it, so nobody will ever freeproc it: the slot is leaked and the table can now hold only 3 processes. Worse, it says Runnable, so a scheduler would pick it and try to run a process that has no page table. (Pid 5 is also used up, which is harmless.)

Fix: undo the claim before returning null (or create the page table before touching the slot):

```
if (*p).pagetable.is_null() {
    (*p).pid = 0;
    (*p).state = ProcState::Unused;
    return ptr::null_mut();
}
```

2. **Context switch (35k).** `Context` is `#[repr(C)]` with 14 `usize` fields, in order `ra`, `sp`, `s0`, …, `s11`. `swtch(old, new)` stores each of those registers into `*old` (`a0`) with `sd`, loads each from `*new` (`a1`) with `ld`, then `ret`s. The test does:

```
init_context(&mut TASK_CTX, task_entry as usize, stack + PGSIZE); // ra = entry, sp = stack top
swtch(&mut SCHED_CTX, &mut TASK_CTX);    // task_entry sets TASK_RAN, then
                                         //   swtch(&mut TASK_CTX, &mut SCHED_CTX)
check(TASK_RAN == 0xABCD);
```

(a) At what byte offset in a `Context` is `s7` stored? Why does `swtch` save `ra`, `sp` and `s0`–`s11`, but not `t0`–`t6` or `a0`–`a7`?

ra is at 0, sp at 8, then sk at 16 + 8k ⟶ s7 at 16 + 56 = (72).

swtch is an ordinary function call, and the calling convention says the caller has already saved any caller-saved registers (t*, a*) it still needs. A function only has to preserve the callee-saved ones, sp and s0–s11. ra is saved too, because it is where the task resumes: those 14 are exactly what "paused here" means.

(b) Kmain's `swtch` runs `ret`. Which code runs next, and on which stack? When does that call to `swtch` finally "return" into `kmain`, and how?

The ret jumps to the ra just loaded from TASK_CTX → the first instruction of task_entry, running on the task's own fresh stack (sp = stack + PGSIZE).

It returns only when task_entry calls swtch(TASK_CTX, SCHED_CTX): that saves the task and loads SCHED_CTX, whose ra is the instruction after kmain's call and whose sp is kmain's stack. Its ret lands back in kmain, which then sees TASK_RAN = 0xABCD.

(c) What happens, and why, if:

(i) the restore half loads from `a0` instead of `a1` (`ld ra, 0(a0)`, …)?

(ii) `init_context` is passed `stack` instead of `stack + PGSIZE`?

(iii) `#[repr(C)]` is removed from `Context`?

(i) It loads back the values it just saved, so swtch is a no-op: ret goes straight back to kmain, the task never runs → "[fail] task never ran".

(ii) sp starts at the bottom of the page. Stacks grow down, so the task's first push writes into the memory below the page, which it does not own: it corrupts whatever is there (another page, or the free list) or faults.

(iii) Rust may reorder the fields, but the assembly still uses fixed offsets 0, 8, 16, … So ld ra, 0(a1) may load some other field (sp, or a 0), ret jumps to garbage and the kernel faults.

3. **Scheduling (36k).** This is the round-robin policy from 36k. After each turn the process yields with `swtch` and the scheduler loop marks it `Runnable` again.

```
fn pick_next(&mut self, states: &[ProcState]) -> Option<usize> {
    let n = states.len();
    (0..n).map(|off| (self.next + off) % n)
          .find(|&i| states[i] == ProcState::Runnable)
          .map(|i| { self.next = (i + 1) % n; i })
}
```

(a) With `self.next = 0`, a 5-slot table starts as

`[A: Runnable, B: Sleeping, C: Runnable, -: Unused, D: Runnable]`

Right after the third turn, `B` wakes up and becomes `Runnable`. Fill in the table for the first 7 turns.

Turn	Index `pick_next` returns	Process that runs	`self.next` after
1	0	A	1
2	2 (skips 1, Sleeping)	C	3
3	4 (skips 3, Unused)	D	0
B wakes up			
4	0	A	1
5	1	B	2
6	2	C	3
7	4 (skips 3, Unused)	D	0

(b) The 36k test runs pids 1, 3, 4 (pid 2 sleeps) for 3 turns each and expects `1 3 4 1 3 4 1 3 4`. What order do you get if `pick_next` sets `self.next = i` instead of `(i + 1) % n`? Why?

`1 1 1 3 3 3 4 4 4`

With self.next = i the next scan starts at the process that just ran, and it is Runnable again, so it is picked again until it retires (Zombie). Only then does the scan move on.

That is the same order as always scanning from 0, which is not round-robin. The interleaved order in the test catches both bugs.

(c) Suppose pid 3's code had an infinite loop before its first yield. What happens to pids 1 and 4, and why? What would fix it?

They starve. pid 1 got its first turn, but once pid 3 is switched in, neither 1 nor 4 ever runs again (pid 4 never runs at all).

Scheduling here is cooperative: the scheduler only gets the CPU back when a process calls swtch to yield. A process that never yields keeps the CPU forever.

Fix: preemption. A timer interrupt fires periodically, traps into the kernel and forces a switch back to the scheduler whether or not the process yields. (The mechanism is still swtch; only what triggers it changes.)